

```
<measurement>[, <tag-key>=<tag-value>...] <field-key>=<field-value>[, fields...] [unix-nano-timestamp]
```

Let us understand what the above format means.

First, the measurement is what we are capturing, i.e., temperature and sound, in our case.

Second, it can have one or more tag values, associated with the reading. For example, we can create a tag name *OfficeName* and provide its value. This will help us use the same database for capturing data across multiple offices and then query the data using the tag values.

Finally, we have one or more key/value pairs, which is the actual value of the measurement. For example, in the case of temperature, it could be value=23 (where 23 is the reading in degree Celsius) or for the sound measurement, it could be value=70 (where 70 is the reading in decibels).

Let us consider our office's name as O1. We can now fire multiple INSERT statements for the temperature measurement as shown below, to simulate some data for our queries:

```
> INSERT temperature,officename=O1 value=23.1
> INSERT temperature,officename=O1 value=23.2
> INSERT temperature,officename=O1 value=23.1
> INSERT temperature,officename=O1 value=23.3
> INSERT temperature,officename=O1 value=23.2
Similarly, we can insert records for the sound measurement as
shown below:
> INSERT sound,officename=O1 value=50.1
> INSERT sound,officename=O1 value=50.2
> INSERT sound,officename=O1 value=55
> INSERT sound,officename=O1 value=50.1
> INSERT sound,officename=O1 value=50.25
```

We can now fire standard SELECT statements to query

our database too, as shown below:

```
> SELECT * FROM temperature
name: temperature
-----
time                                officename  value
1475390293075674300                01          23.1
1475390298980348300                01          23.2
1475390309734637600                01          23.1
1475390302341835800                01          23.3
1475390304245307200                01          23.2

> SELECT * FROM sound
name: sound
-----
time                                officename  value
1475390867994939100                01          50.1
1475390870004235200                01          50.2
1475390872600605800                01          55
1475390876404970300                01          50.1
1475390879424646200                01          50.25
>
```

This is just a fraction of the capability of the InfluxDB DDL and DML statements. Take a look at https://docs.influxdata.com/influxdb/v1.0/query_language/spec/ for the entire specifications.

InfluxDB Web administration application

The InfluxDB database ships with a default Web application, which you can use to query the databases and also to write data into the database. The default Web server is running on port 8083 of your InfluxDB host machine. In our case it is *localhost*.

Launch a browser and visit <http://localhost:8083>. This will bring up the admin UI, as shown in Figure 1.

You can also execute any of the queries for your respective databases. In our case, we select the database *OfficeMonitoringDB* from the Database list at the top right, and then fire a SELECT statement as shown in Figure 2.

You will also notice a 'Write Data' option in the top menu that you can use to insert records into the database. This is similar to what we did earlier with the InfluxDB client application.

The REST API

InfluxDB exposes a REST API on port 8086. This API exposes three endpoints -- */ping*, */query* and */write*, and it is clear from their names that they can

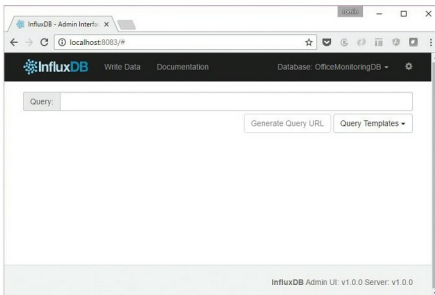


Figure 1: InfluxDB admin Web application